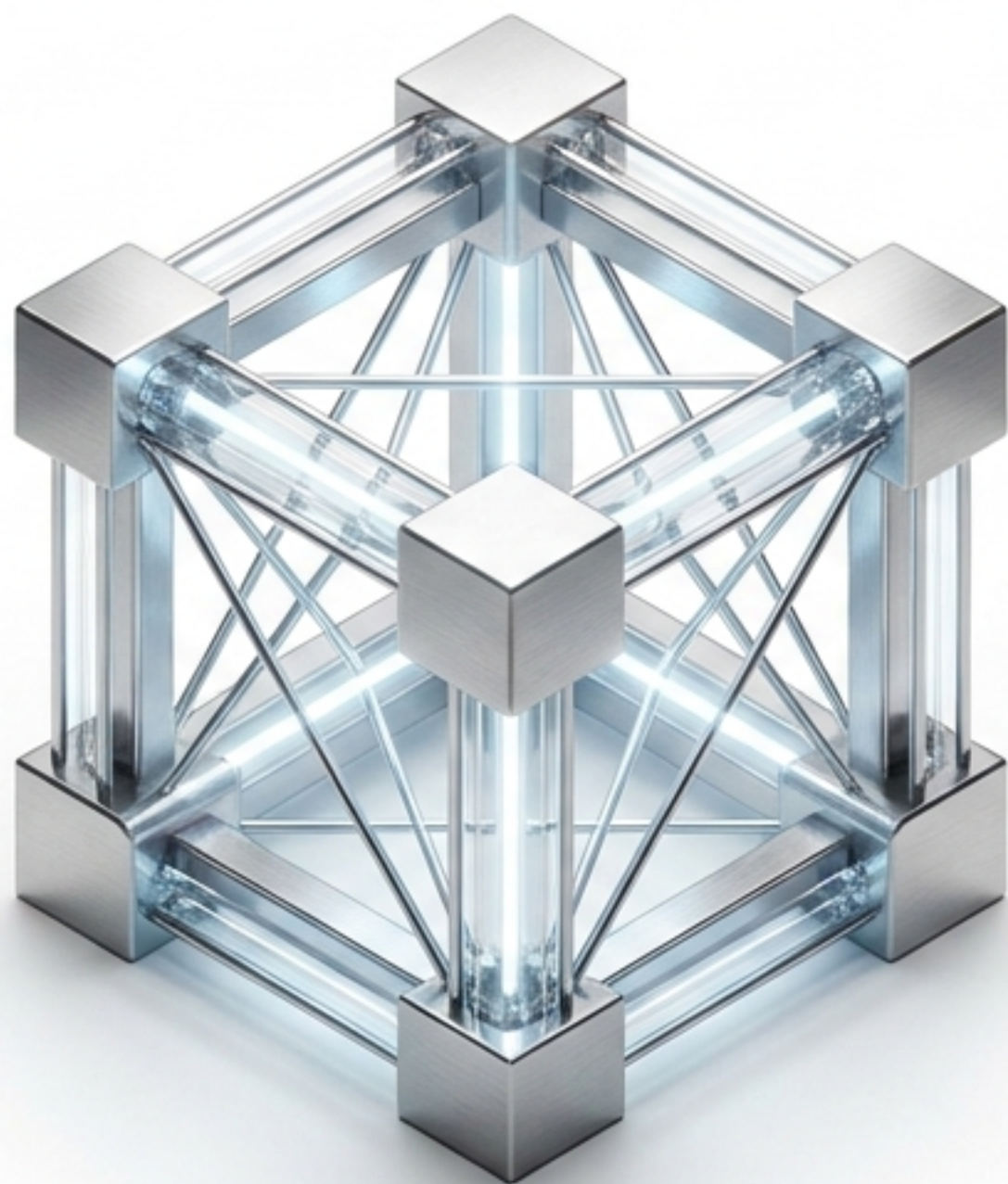




Lập trình Chuyên sâu với Keras

Học Sâu (Deep Learning)

Chương 7

Đại học Đông Á

August 14, 2026

Lộ trình Kiến trúc Mô hình Học sâu



1. Thiết lý Thiết kế của Keras



2. Ba phương pháp Xây dựng Mô hình



3. Tùy biến Vòng lặp Huấn luyện (Customizing)



4. Callbacks: Tự động hóa quá trình Huấn luyện



5. Giám sát với TensorBoard



6. Tổng kết

Từng bước Phơi bày Sự phức tạp (Progressive Disclosure)

Nguyên tắc Keras

Làm cho mọi thứ trở nên dễ bắt đầu, nhưng vẫn hoàn toàn khả thi để xử lý các bài toán siêu phức tạp.

Phổ quy trình

Không có một “con đường duy nhất”. Keras cung cấp một Phổ quy trình (Spectrum of workflows). Các bài toán đơn giản chỉ cần vài dòng mã, công việc phức tạp cho phép viết mã chi tiết.

Sequential API
+ built-in layers

Functional API
+ built-in layers

Functional API
+ custom layers
+ custom metrics
+ custom losses
+ ...

Subclassing:
write everything
yourself from scratch

Novice users,
simple models

Engineers with
standard use
cases

Engineers with
niche use cases
requiring bespoke
solutions

Researchers

Hình 7.1: Từ Sequential đến Model Subclassing

Thiết kế Đa mô hình trong Hệ sinh thái Keras

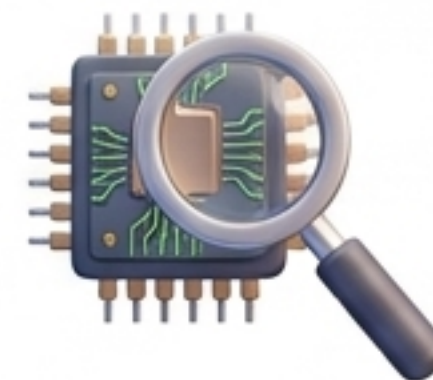


Người mới (Beginner)

Sử dụng Keras như Scikit-learn.
Dành cho mô hình đơn giản,
tận dụng các hàm có sẵn như
`model.fit()` và `Sequential`.



Giao tiếp trơn tru



Nhà nghiên cứu (Researcher)

Sử dụng Keras như NumPy
/ TensorFlow lõi. Viết tay
mọi thứ từ đầu bằng
`GradientTape`.

Đặc biệt: Các API này giao tiếp trơn tru với nhau. Một mô hình `Sequential` hoàn toàn có thể được lồng vào bên trong một mô hình `Subclassing`.

Vòng lặp Tiến bộ trong Nghiên cứu Khoa học

Khoa học là quá trình thử nghiệm lặp đi lặp lại (Iterative Process). Mục tiêu của Keras là tối ưu hóa UX để thu ngắn tối đa thời gian lặp trình.



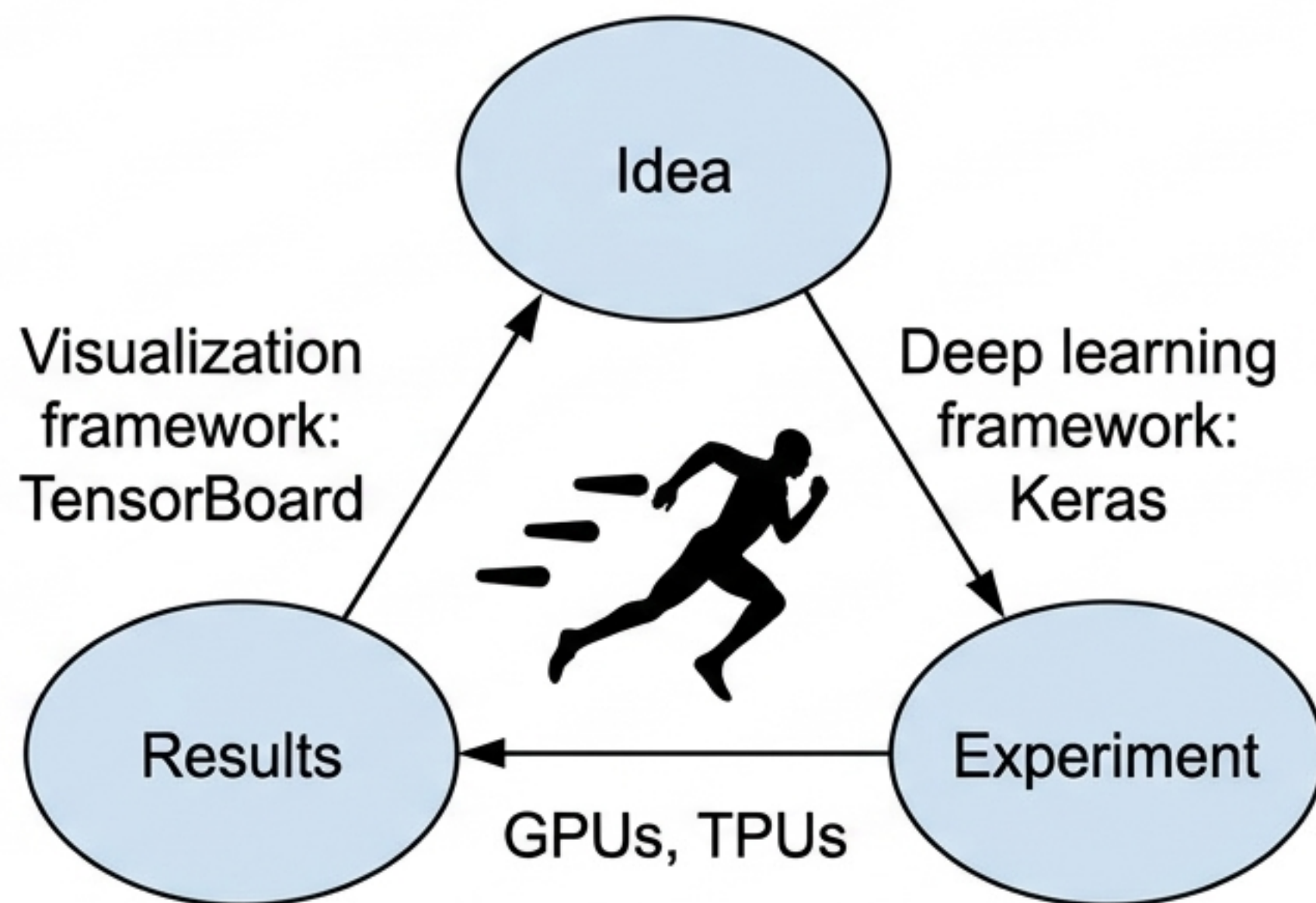
1. Idea: Khởi tạo ý tưởng.



2. Experiment: Cài đặt mã nguồn.



3. Evaluate: Đánh giá kết quả
→ Rút ra bài học và tạo Idea mới.



Hình 7.2: Vòng lặp phát triển Mô hình ML

Phương pháp 1: Mạng Tuyến tính Sequential API

Đặc điểm

- Bản chất là cấu trúc dữ liệu dạng Danh sách (Python List).
- Đơn giản nhất, dễ tiếp cận nhất.
- Các lớp được xếp chồng tuyến tính (lớp sau nối tiếp ngay sau lớp trước).



Hạn chế cốt lõi

- Chỉ áp dụng được cho cấu trúc: 1 đầu vào, 1 đầu ra.
- KHÔNG thể áp dụng cho: Mô hình đa đầu vào (Multi-input), đa đầu ra (Multi-output), hoặc cấu trúc phân nhánh.

Khởi tạo Mạng Sequential trong Thực tế

Cách 1: Khởi tạo bằng Danh sách (List)

```
import keras
from keras import layers

model = keras.Sequential([
    layers.Dense(64, activation="relu"),
    layers.Dense(10,
activation="softmax"),
])
```

Cách 2: Khởi tạo dần dần bằng add()

```
model = keras.Sequential()

model.add(layers.Dense(64,
activation="relu"))
model.add(layers.Dense(10,
activation="softmax"))
```

Cơ chế Khởi tạo Trọng số và Kích thước (Shape)

Quy tắc cốt lõi: Các lớp trong Keras chỉ tạo trọng số (build) khi biết chính xác kích thước đầu vào (input shape).

Step 1: Trạng thái ban đầu (Before build)

```
>>> model.weights  
[]
```

Step 2: Kích hoạt **Build**

```
>>> model.build(input_shape=(None, 3))
```

Step 3: Hình thành Trọng số (Weights created)

```
>>> model.weights  
[<Variable shape=(3, 64)...>, <Variable shape=(64,...)>]
```

Kiểm tra cấu trúc

```
>>> model.summary()  
Model: "sequential_1"  
...
```

Kỹ thuật Khai báo Input Shape Từ đầu



Sử dụng đối tượng **Input** để khai báo đầu vào, cho phép toàn bộ mô hình được **build** tự động ngay từ thời điểm khởi tạo.

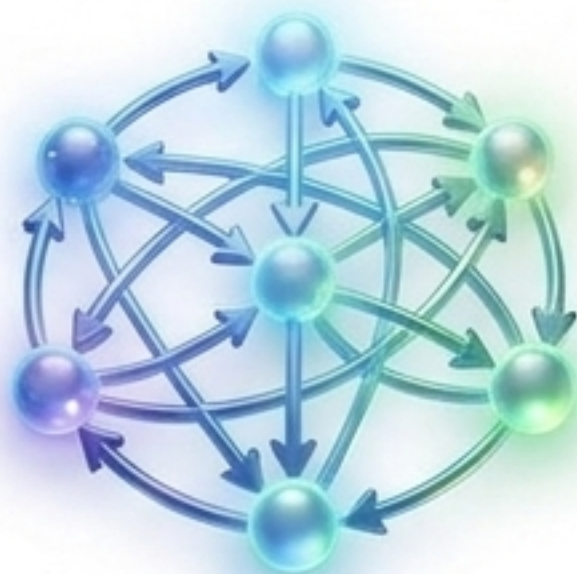
```
model = keras.Sequential()  
model.add(keras.Input(shape=(3,)))  
model.add(layers.Dense(64, activation="relu"))  
  
# Model is built automatically, we can call summary now  
model.summary()
```

Thủ thuật này đặc biệt hữu ích và an toàn khi thiết kế các mạng CNN sâu.

Phương pháp 2: Sự linh hoạt của **Functional API**



Coi các layer là **các Hàm (Functions) độc lập**.
Chúng nhận Tensor và trả về Tensor.



Cung cấp giải pháp **linh hoạt** tuyệt đối, cho phép xây dựng mạng **Đồ thị đa hướng** không chu trình (Directed Acyclic Graph).



Đây là API **phổ biến nhất** trong thực tế công nghiệp.
Mang lại cảm giác như chơi lắp ráp LEGO.



Hỗ trợ **hoàn hảo** cho việc định nghĩa nhiều luồng vào ra và chia nhánh đồ thị.

Bốn Bước Xây dựng Functional API

```
# 1. Define Input
inputs = keras.Input(shape=(3,), name="my_input")

# 2. Build graph by calling layers like functions
features = layers.Dense(64, activation="relu")(inputs)

# 3. Create outputs
outputs = layers.Dense(10, activation="softmax")(features)

# 4. Initialize Model
model = keras.Model(inputs=inputs,
                    outputs=outputs,
                    name="my_functional_model")
```

1

Định nghĩa **Input**.

Chú ý: Đối tượng inputs là một Symbolic Tensor, chỉ chứa thông tin shape và dtype, không chứa dữ liệu.

2

Gọi layer như các hàm để tạo Đồ thị.

3

Định nghĩa **Output**.

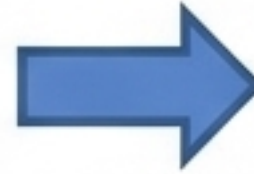
4

Khởi tạo **Mô hình**.

Tình huống Thực tế: Hệ thống Phân loại Vé (Ticket Classifier)

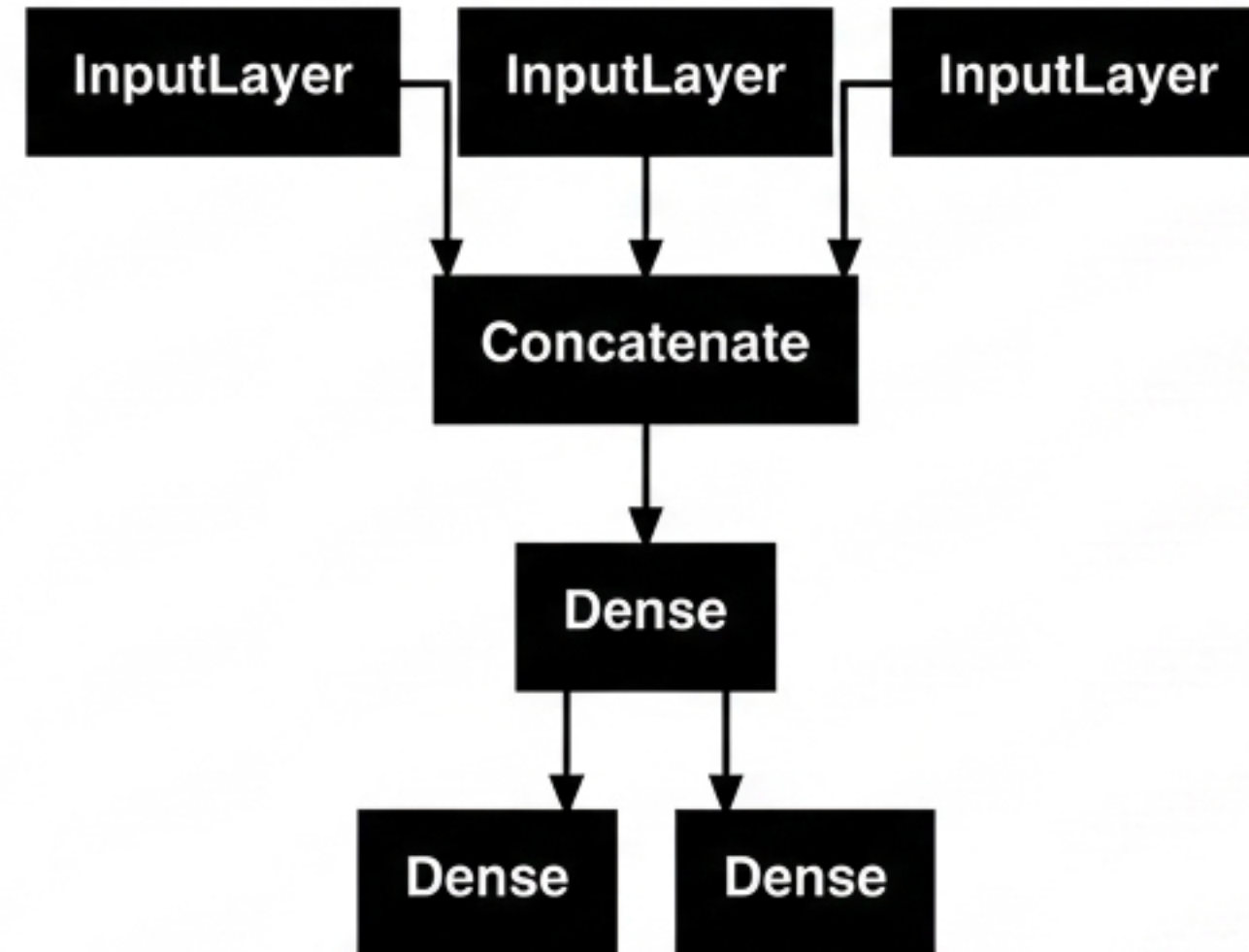
3 Đầu vào (Inputs):

Tiêu đề (title) | Nội dung (text_body) |
Thẻ (tags)



2 Đầu ra (Outputs):

Mức độ ưu tiên (priority: 0 đến 1) |
Phòng ban xử lý (department)



Hình 7.3: Sơ đồ mạng đa luồng phân loại vé

Cài đặt Mã nguồn: Hệ thống Phân loại Vé

```
vocabulary_size = 10000; num_tags = 100; num_departments = 4

# Inputs
title = keras.Input(shape=(vocabulary_size,), name="title")
text_body = keras.Input(shape=(vocabulary_size,), name="text_body")
tags = keras.Input(shape=(num_tags,), name="tags")

# Concatenate features
features = layers.Concatenate()([title, text_body, tags])
features = layers.Dense(64, activation="relu", name="dense_features")(features)

# Outputs
priority = layers.Dense(1, activation="sigmoid", name="priority")(features)
department = layers.Dense(num_departments, activation="softmax", name="department")(features)

# Initialize Model
model = keras.Model(inputs=[title, text_body, tags], outputs=[priority, department])
```

Huấn luyện Mô hình Đa Luồng (Multi-Input/Output)

Khi sử dụng **fit()**, Keras hỗ trợ truyền dữ liệu dưới dạng List hoặc Dictionary.

```
model.compile(optimizer="adam",
              loss={"priority": "mean_squared_error",
                   "department": "sparse_categorical_crossentropy"},
              metrics={"priority": ["mae"], "department": ["accuracy"]})

model.fit(
    {"title": title_data, "text_body": text_body_data, "tags": tags_data},
    {"priority": priority_data, "department": department_data},
    epochs=1)
```



Best Practice

Lời khuyên thiết kế: Sử dụng tham số name của Input/Output (Dictionary mode) luôn dễ đọc, dễ kiểm soát và dễ bảo trì hơn rất nhiều so với việc truyền mảng vô danh (List mode).

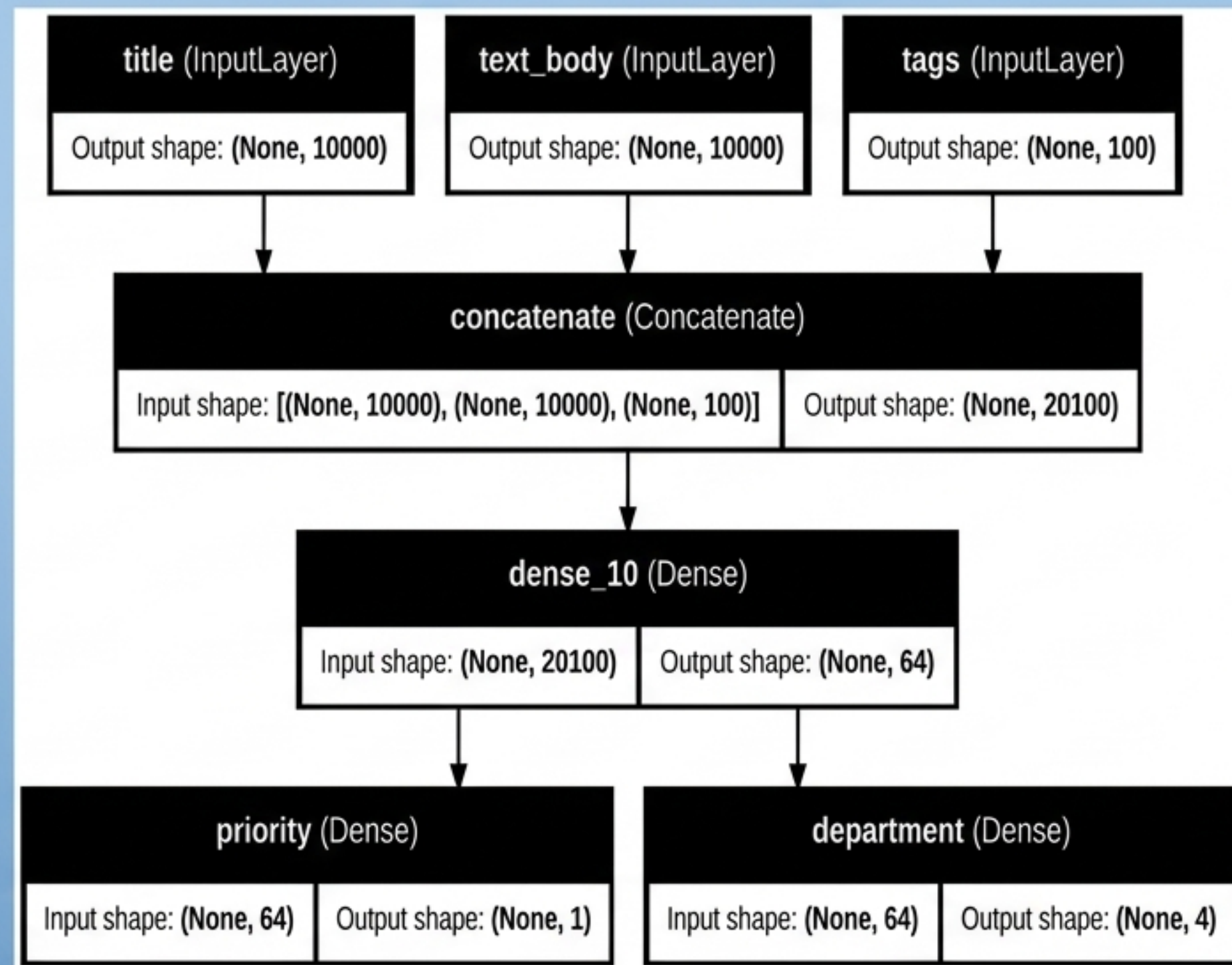
Giám sát Luồng dữ liệu và Kiểm soát Hình dáng (Shapes)

Keras cung cấp tiện ích `plot_model` để trực quan hóa sơ đồ mạng.

Bật tham số `show_shapes=True` để in ra chi tiết kích thước luồng dữ liệu.

Lưu ý đọc chỉ số: `TensorShape (None, 10000)` - Ký tự `None` đại diện cho kích thước Lô (Batch size) có thể thay đổi linh hoạt.

Tác dụng lỗi: Việc kiểm tra kích thước ngay tại bước thiết kế (Design Time) giúp loại bỏ triệt để các lỗi ma trận kích thước lệch pha (Shape Mismatch).



Hình 7.4: Đồ thị truyền tải Shapes